

Special Participation B -- Gemini Pro on HW 8, Arvind Kruthiventy

Executive Summary

In this report, I utilized Gemini Pro model for completing the SSM coding portions of Homework 8. I inputted each portion of the coding question separately and checked the model's outputs but it appeared to nearly one-shot all the portions and provided clean, formatted code with very detailed explanations. It made a small mistake in implementing the convolution forward pass as it treated the operation as a traditional convolution, but with a prompt from me, it immediately identified its mistake. Interesting, its SSM convolution forward implementation seems slightly more efficient compared to the provided solutions. The analysis of its own code were thorough and a standout as it makes it easy to understand how lines of code correspond to the more abstract details. However, there is a clear tradeoff as the Pro model takes a significant amount of time even for the simpler questions.

User

SSM Update Rule

We consider an Linear RNN described by the update:

$$h_{t+1} = W h_t + U x_t + b$$

for $t = 0, 1, \dots, T - 1$. The variables are:

- $h_t \in R^H$, the hidden state at time t .
- $x_t \in R^{N \times D}$, the input at time t .
- $W \in R^{H \times H}$, the recurrent weight matrix.
- $U \in R^{H \times D}$, the input projection matrix.
- $b \in R^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H .

The code to fill in is below:

Python

```

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
    Returns:
        h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
    """
    #####
    #####
    #                                TODO: Implement the recurrent pass here
    #
    #####
    #####

    #####
    #####
    #                                END OF YOUR CODE
    #
    #####
    #####
    return h_all

```

Assistant

Here is the completed code to implement the unrolled forward pass of the linear RNN.

The core of the implementation is to loop through the time dimension T and apply the recurrent update rule at each step.

The Logic Explained

1. **Dimensions & Batching:** The input x has shape (N, T, D) (Batch, Time, InputDim) and the desired output h_{all} has shape (N, T, H) (Batch, Time, HiddenDim). This "batch-first" convention is standard.

2. **Initialization:** We must initialize the first hidden state h_0 to a zero vector. Since we're processing a batch, this will be a tensor of zeros with shape (N, H) . We also pre-allocate the output tensor `h_all` to store the results.
 3. **The Loop:** We iterate T times. In each step t of the loop:
 - We get the current input slice x_t , which has shape (N, D) .
 - We use the *previous* hidden state h_t (which has shape (N, H)) to compute the *next* hidden state h_{t+1} .
 4. **Matrix Math:** The math equation is $h_{t+1} = Wh_t + Ux_t + b$.
 - This is written for column vectors (e.g., h_t is $H \times 1$).
 - Our code uses row vectors in a batch (e.g., `h_t` is $N \times H$).
 - The equivalent "batched row-vector" operation for Wh_t is `h_t @ W.T`.
 - `h_t (N, H) @ W.T (H, H) (N, H)`
 - Similarly, the operation for Ux_t is `x_t @ U.T`.
 - `x_t (N, D) @ U.T (D, H) (N, H)`
 - The bias `b (H,)` is automatically broadcast across the batch `N`.
 5. **Storing Output:** The new state `h_t` (which represents h_{t+1} in the math) is stored in the `t`-th position of `h_all`.
-

Here is the code to fill in the function:

Python

```
#####
##
#                                     TODO: Implement the recurrent pass here
#
#####
##

# We assume a library like NumPy is imported, e.g., as 'np'
# or that x is a PyTorch/JAX tensor. The logic is the same.
# Get the library's 'zeros' function from the input tensor
# (This is a robust way to handle any framework)
try:
    # PyTorch/JAX-like
    zeros = x.new_zeros
except AttributeError:
    # NumPy-like
```

```

import numpy as np
zeros = lambda size, dtype: np.zeros(size, dtype=dtype)

# Get dimensions
N, T, D = x.shape
H = b.shape[0] # Hidden dimension

# Pre-allocate the output tensor for all hidden states
h_all = zeros((N, T, H), dtype=x.dtype)

# Initialize h_0 (the hidden state at t=0)
h_t = zeros((N, H), dtype=x.dtype)

# Loop over the time steps
for t in range(T):
    # Get the input for the current time step
    x_t = x[:, t, :] # Shape (N, D)

    # Apply the linear RNN update rule
    #  $h_{t+1} = W h_t + U x_t + b$  (math notation)
    #  $h_t = h_t @ W.T + x_t @ U.T + b$  (batched code equivalent)
    h_t = h_t @ W.T + x_t @ U.T + b

    # Store the new hidden state ( $h_{t+1}$ )
    h_all[:, t, :] = h_t

#####
##
#
#
#
#
#####
##

```

Annotation:

This question is relatively simple so gemini one-shots it and produces a working solution with no additional details except the problem description. The implementation is easily understood because of the extensive comments and matches the solution provided by TA's. One thing is that it didnt understand that imports were already done even when it was provided in subsequent comments.

User

the assignment is in pytorch so this and all other code should be pytorch

Assistant

Got it. Here is the PyTorch-specific implementation for the function.

This version uses `torch.nn.functional.linear` which is the idiomatic way to perform the $= x^T + b$ operation in PyTorch.

PyTorch Implementation

Here is the code to fill in the function, assuming `torch` is imported.

Python

```
import torch
import torch.nn.functional as F

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        
$$h_{t+1} = W h_t + U x_t + b$$

    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps

    Returns:
        h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
    """

#####
##
#           TODO: Implement the recurrent pass here
#
#####
##
```

```

# Get dimensions from input tensors
N, T, D = x.shape
H = W.shape[0] # Hidden dimension

# Get device and dtype from the input tensor 'x' to ensure all
# new tensors are created on the same device (e.g., CPU or GPU)
device = x.device
dtype = x.dtype

# Pre-allocate the output tensor for all hidden states
h_all = torch.zeros((N, T, H), device=device, dtype=dtype)

# Initialize h_0 (the hidden state at t=0)
h_t = torch.zeros((N, H), device=device, dtype=dtype)

# Loop over the time steps
for t in range(T):
    # Get the input for the current time step
    x_t = x[:, t, :] # Shape (N, D)

    # Apply the linear RNN update rule.
    # F.linear(input, weight) calculates input @ weight.T

    #  $h_{t+1} = h_t @ W.T + x_t @ U.T + b$ 
    h_t = F.linear(h_t, W) + F.linear(x_t, U) + b

    # Store the new hidden state ( $h_{t+1}$ )
    h_all[:, t, :] = h_t

#####
##
#                                     END OF YOUR CODE
#

#####
##
    return h_all

```

User

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

- `make_conv_kernel(W, T)` : This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel . Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)` : This function will take weights W, U, b and input x and return the hidden states h across different time steps.

Python

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
    implementing
    the ssm forward pass using conv1d.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """
    #####
    #####
    #                                TODO: Implement the kernel here
    #
    #####
    #####

    #####
    #####
    #                                END OF YOUR CODE          S
    #
    #####
    #####
    return kernel_for_conv
```

Assistant

Here is the PyTorch implementation for `make_conv_kernel`.

This solution uses the "divide and conquer" (or "doubling") strategy suggested by the problem. Since T is a power of 2, we can compute the powers of W efficiently.

The Logic Explained

1. **Goal:** We need to create a kernel tensor of shape (H, H, T) where $t = W^t$ for $t = 0, \dots, T - 1$. So the kernel is the stack $W, W, \dots, W^{T-1}$.
2. **Naive Approach:** We could use a `for` loop from $t = 1$ to $T - 1$ and compute $t = W \cdot t_{-1}$. This is T in matrix multiplications.
3. **Divide and Conquer (Doubling) Approach:** We can do this in T steps.
 - **Start (k=1):**
 - `powers = (shape H, H, 1)`
 - `w_pow = W (shape H, H)`
 - **Step 1 (k=2):**
 - We already have `powers`. We need W .
 - Compute the next block: $W^2 = W \cdot W$.
 - Concatenate: `powers = , W (shape H, H, 2)`.
 - Update `w_pow` for the *next* step: $W^2 \cdot W = W^3$.
 - **Step 2 (k=4):**
 - We already have `powers`, W . We need W^2, W^3 .
 - Compute the next block: $W^4, W^5 = W^2 \cdot W^2, W^2 \cdot W^3$.
 - Concatenate: `powers = , W, W, W (shape H, H, 4)`.
 - Update `w_pow`: $W^4 \cdot W = W^5$.
 - **Step 3 (k=8):**
 - We have `powers`, W . We need $W^4, \dots, W^7$.
 - Compute the next block: $W^8, \dots, W^9 = W^4 \cdot W^4, \dots, W^4 \cdot W^7$.
 - Concatenate: `powers = , , , W (shape H, H, 8)`.
 - Update `w_pow`: $W^8 \cdot W = W^9$.
 - We repeat this T times until we have the full kernel.

Here is the code to fill in the function:

Python


```

import torch

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
    implementing
    the ssm forward pass using conv1d.

    The kernel K should be structured as:
    K[:, :, t] = W^t
    So K = [I, W, W^2, ..., W^{T-1}]

    Args:
        W: (H, H) weight matrix
        T: scalar, the sequence length (must be a power of 2)

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """

    #####
    ##
    #                                TODO: Implement the kernel here                                #
    #####
    ##

    H = W.shape[0]
    device = W.device
    dtype = W.dtype

    # Initialize the kernel with the 0-th power: W^0 = I
    # We start with k=1 (we have 1 power)
    kernel_for_conv = torch.eye(H, device=device, dtype=dtype).unsqueeze(-1)
    # Shape (H, H, 1)

    # This will store W^k (W^1, W^2, W^4, W^8, ...)
    W_power = W

    k = 1
    # Loop log2(T) times
    while k < T:
        # At the start of this loop:
        # kernel_for_conv has shape (H, H, k) and holds [I, ..., W^{k-1}]
        # W_power has shape (H, H) and holds W^k

```

```

# 1. Compute the next k powers: [W^k, W^{k+1}, ..., W^{2k-1}]
#   This is computed as W^k @ [I, W, ..., W^{k-1}]
#   (H, H) @ (H, H, k) -> (H, H, k)
next_k_powers = torch.matmul(W_power, kernel_for_conv)

# 2. Concatenate to get [I, ..., W^{2k-1}]
#   (H, H, k) cat (H, H, k) on dim=2 -> (H, H, 2k)
kernel_for_conv = torch.cat([kernel_for_conv, next_k_powers], dim=2)

# 3. Update W_power for the next iteration: (W^k) @ (W^k) -> W^{2k}
W_power = torch.matmul(W_power, W_power)

# 4. Update k
k = k * 2

#####
##
#                               END OF YOUR CODE
#

#####
##
    return kernel_for_conv

```

Annotation:

Gemini again gives a good detailed output explanation that breaks down the ssm conv kernel creation process in easily understandable steps that correspond directly to lines in code. This makes it very easy to extrapolate the algorithm details to the implementation details. One interesting thing is that it defaults to the faster implementation of the kernel creation (the one thats similar to the make_conv_kernel_fast in the provided solutiions) with no explicit mention or guidance towards speed. Another interesting quirk is that it treats the convolutional kernel in the traditional sense: it needs to be flipped with respect to the outputs to perform the convolution. As will be seen in the later ouputs, it technically one-shot this portion as well with just the problem description.

User

Python

```

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    #####
    ##
    #                                TODO: Implement the convolution here
    #

    #####
    ##

    #####
    ##
    #                                END OF YOUR CODE
    #

    #####
    ##
    return h_all

```

Here is the completed PyTorch implementation for `conv_ssm_forward`.

The Logic Explained

1. **Goal:** We want to compute $h_{t+1} = \sum_{t=0}^t W^{t-} \cdot x_t$, where $x_t = Ux + b$. This is a convolution.
2. **PyTorch `conv1d`:** The function `torch.nn.functional.conv1d` actually computes a **cross-correlation**, not a convolution.
 - Convolution: $t = t -$
 - Cross-Correlation: $t = t +$
3. **The Trick:** To compute a convolution using a cross-correlation function, you must **flip the kernel**.
 - We have our kernel $= W, \dots, W^{T-1}$ from `make_conv_kernel`.
 - We flip it along the time dimension (dim 2) to get $= W^{T-1}, \dots, W, .$
4. **Causality & Padding:** The sum $h_{t+1} = \sum_{t=0}^t \dots$ is *causal* (the output at time t only depends on inputs up to time t).
 - The `conv1d` operation will have a kernel of size T . To make the convolution causal and get T output steps, we must pad the input with $T - 1$ zeros on the **left**.
 - `s` (Input): `(N, H, T)`
 - `s_padded` (Padded Input): `F.pad(s, (T-1, 0))` Shape `(N, H, 2T - 1)`
5. **Running the Convolution:**
 - `input: s_padded` (Shape `N, H, 2T - 1`)
 - `weight: kernel_flipped` (Shape `H, H, T`)
 - `h_conv = F.conv1d(s_padded, kernel_flipped)`
 - The output `h_conv` will have length $= - - 1 = T - 1 - T - 1 = T$.
 - The output shape will be `(N, H, T)`.
6. **Final Permutation:** The function's docstring specifies a return shape of `(N, T, H)`. The setup code permuted the input `s` from `(N, T, H)` to `(N, H, T)`. We must now permute the output `h_conv` back.

Here is the code to fill in the function:

Python

```
import torch
import torch.nn.functional as F

# (Assuming make_conv_kernel is defined above)

def conv_ssm_forward(W, U, b, x):
```

```

"""
Convolution-based forward pass for a batch of sequences.

RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,) bias
    x: (N, T, D) input (batch=N, time steps=T, input dim=D)

Returns:
    h_all: (N, T, H) hidden states
"""
N, T, D = x.shape
H = W.shape[0]

# s is the pre-computed input term  $s_t = U x_t + b$ 
# s has shape (N, T, H)
s = x @ U.T + b

# Permute s to (N, H, T) to match conv1d's expected
# input format (N, C_in, L_in)
s = s.permute(0, 2, 1)

# Build the kernel with shape (H, H, T).
kernel = make_conv_kernel(W, T)

#####
##
#
#
#####
##

# 1. Flip the kernel for convolution
# F.conv1d computes cross-correlation. To get convolution,
# we must flip the kernel's time dimension (dim 2).
kernel_flipped = torch.flip(kernel, dims=[2])

# 2. Pad the input s for causal convolution
# We pad on the LEFT with (T-1) zeros.
# s shape is (N, H, T)
s_padded = F.pad(s, (T - 1, 0))

```

```

# s_padded shape is (N, H, 2T - 1)

# 3. Apply 1D convolution
# input: (N, C_in, L_in) = (N, H, 2T - 1)
# weight: (C_out, C_in, K) = (H, H, T)
# output: (N, C_out, L_out) where L_out = (2T - 1) - T + 1 = T
# h_conv shape is (N, H, T)
h_conv = F.conv1d(s_padded, kernel_flipped)

# 4. Permute the output
# We need to return (N, T, H) as specified in the docstring.
h_all = h_conv.permute(0, 2, 1)

#####
##
#
#
#
#
#####
##
return h_all

```

Annotation:

Again as above, the output is very detailed and provides step by step analysis which contrasts other LLMs in how specific its answers are. It had a subtle one line mistake as will be explored in the later responses; this was checked by the sanity check and was rooted partially by the fact that it implemented the convolutional kernel as a traditional convolution.

User

I think your make conv kernel implementation is incorrect because your implementations of the rnn based ssm forward and cnn based ssm forward disagree by a large margin

Assistant

You are absolutely right to point that out. A disagreement by a large margin means there is a logical flaw, not just a floating-point error.

After re-checking my logic, I found a critical ambiguity between the problem's mathematical definition and the standard PyTorch implementation. This ambiguity leads to two different valid models, and my two code snippets were implementing *different models*.

Here is the breakdown of the error:

The Core Mismatch

1. **The Math:** The problem states the RNN update rule is $h_{t+1} = h_t + x_t + b$. This is a "column-vector" convention, where h_t is a column vector of shape $(H, 1)$.
2. **The `unrolled_ssm_forward` Code:** This function uses standard PyTorch "row-vector" convention.
 - `h_t` has shape (N, H) .
 - The code `F.linear(h_t, W)` computes $h_t @ W.T$.
 - This means my `unrolled_ssm_forward` code actually implemented the recurrence:
$$h_{Tt+1} = h_{Tt} + x_{Tt} + b^T$$
 - Notice that the matrix being used in the recurrence is W^T , not W .
3. **The `conv_ssm_forward` Code:**
 - The skeleton *you provided* explicitly calls `kernel = make_conv_kernel(W, T)`.
 - My `make_conv_kernel` function (correctly) builds a kernel from powers of $x_t = x_0, \dots, x_{T-1}$.
 - My convolution logic then (correctly) implements the *column-vector* recurrence
$$h_{t+1} = h_t + \dots$$

This is the error:

- `unrolled_ssm_forward` was implementing a model based on W^T .
- `conv_ssm_forward` was implementing a model based on W .

These two will, of course, "disagree by a large margin."

The Fix

To make the two implementations agree, the convolution-based method must also be based on W^T . The fix is to **call `make_conv_kernel` with $W.T$ instead of W** .

The `make_conv_kernel` function itself is correct as written, but the *caller* (`conv_ssm_forward`) was using it in a way that was inconsistent with the `unrolled_ssm_forward` implementation.

Here is the corrected `conv_ssm_forward` function. The only change is in the line where `kernel` is created.

Python

```
def conv_ssm_forward(W, U, b, x):  
    """  
    Convolution-based forward pass for a batch of sequences.  
    ...
```

```
"""
N, T, D = x.shape
H = W.shape[0]

s = x @ U.T + b
s = s.permute(0, 2, 1)

#####
##
#                               THE FIX IS ON THIS LINE
#

#####
##

# Build the kernel with powers of W.T to match the unrolled version
kernel = make_conv_kernel(W.T, T)

#####
##
#                               TODO: Implement the convolution here
#

#####
##

# 1. Flip the kernel for convolution
kernel_flipped = torch.flip(kernel, dims=[2])

# 2. Pad the input s for causal convolution
s_padded = F.pad(s, (T - 1, 0))

# 3. Apply 1D convolution
h_conv = F.conv1d(s_padded, kernel_flipped)

# 4. Permute the output
h_all = h_conv.permute(0, 2, 1)

#####
##
#                               END OF YOUR CODE
#
```



```
#####  
##  
    return h_all
```

Annotation:

It immediately identified its mistake which was missing the transpose and provided a specific set of consistent reasons for that mistake. However it fixes `conv_ssm_forward` and not `make_conv_kernel` which could have also been changed to get the right behavior. When comparing gemini's response with the provided solutions Gemini's response is off from the `make_conv_kernel_fast` by one line when it concatenates in opposite order. However Gemini doesn't change this function but resorts to transposing the weights.

User

Question 2

What is the number of operations being performed in the recurrence based implementation of the SSM forward pass?

Assistant

Here is a breakdown of the operations for the recurrence-based implementation.

The total number of floating-point operations (FLOPs) is dominated by the matrix multiplications inside the time-step loop.

Operations per Time Step (t)

The core update rule, $h_{t+1} = Wh_t + Ux_t + b$, is computed for a batch N at each step t . In PyTorch, this is:

```
h_t = F.linear(h_t, W) + F.linear(x_t, U) + b
```

Which is equivalent to:

```
h_t = (h_t @ W.T) + (x_t @ U.T) + b
```

Let's count the FLOPs for each part (approximating 1 MAC = 2 FLOPs):

1. Recurrent term ($h_t @ W.T$):

- h_t shape: (N, H)
- $W.T$ shape: (H, H)
- Operations: $\times N \times H \times H = NH$ FLOPs

2. Input term ($x_t @ U.T$):

- x_t shape: (N, D)
- $U.T$ shape: (D, H)
- Operations: $\times N \times D \times H = NDH$ FLOPs

3. Additions (Term 1 + Term 2 + bias):

- Adding the two resulting (N, H) matrices: $N \times H$ FLOPs
- Adding the $(H,)$ bias (broadcasted): $N \times H$ FLOPs
- Total Additions: NH FLOPs

Total FLOPs per step (t):

$$NH + NDH + NH = NHH + D + 1$$



Total Operations

The recurrence loop runs T times.

$$\text{Total FLOPs} = T \times$$

$$\text{Total FLOPs} = T \times NH + NDH + NH$$

$$\text{Total FLOPs} = NTH + NTDH + NTH$$



Asymptotic Complexity

In Big-O notation, we drop the lower-order terms and constants. The complexity is:

$$NTH + NDH$$

This shows that the cost is **linear** in the batch size N and sequence length T . It is **quadratic** in the hidden state size H .

Annotation:

It one shots this question and provides a detailed breakdown on the individual components that contribute to the computation. There's no provided solution for this or the following question 3, but the reasoning presented makes sense and agrees with my reasoning for this part as well.

User

Question 3

What is the number of operations being performed in the convolution based implementation of the SSM forward pass?

Assistant

Here is the breakdown of the operations for the convolution-based implementation.

The total cost is the sum of three main parts:

1. **Input Projection:** Calculating $t = Ux_t + b$ for all t .
 2. **Kernel Creation:** Running `make_conv_kernel` to get `.`
 3. **The Convolution:** Running `F.conv1d`.
-



Input Projection

This part computes `s = x @ U.T + b`.

- **Operation:** A batched matrix multiplication. The `(N, T, D)` input `x` is multiplied by the `(D, H)` transposed matrix `U.T`.
 - **Dimensions:** This is equivalent to `(N * T, D) @ (D, H)`.
 - **FLOPs:** $\times N \times T \times D \times H = NTDH$.
 - **Bias Add:** Adds $N \times T \times H$ operations.
 - **Complexity:** $NTDH$
-



Kernel Creation

This part computes `kernel = make_conv_kernel(W, T)`.

- **Operation:** This function uses a "doubling" method that runs for T steps.
- **Analysis:**
 - The loop computes `next_k_powers = torch.matmul(W_power, kernel_for_conv)`.
 - In the i -th iteration (where i goes from 0 to $T - 1$), `kernel_for_conv` has $= 2^i$ powers.

- The `matmul` is a batched $(H, H) @ (H, H, k)$, which costs $\times H$ FLOPs.
 - The total cost is the sum: $\sum_{t=1}^T \times H = H \sum_{t=1}^T$.
 - This geometric series sums to $H^T - 1 = HT - 1$.
 - We also have the `W_power = W_power @ W_power` update in each step, which adds $H T$.
 - **Complexity:** The dominant term is TH .
-

The Convolution

This is the main computation: `h_conv = F.conv1d(s_padded, kernel_flipped)`.

This is a **direct, time-domain convolution**.

- **Input Shape:** $(N, C_{in}, L_{in}) = (N, H, 2T - 1)$
- **Kernel Shape:** $(C_{out}, C_{in}, K) = (H, H, T)$
- **Output Shape:** $(N, C_{out}, L_{out}) = (N, H, T)$

The standard formula for FLOPs in a `conv1d` is:

$$\times N \times \times \times \times$$

Let's substitute our dimensions:

- $\times N \times H \times H \times T \times T$
 - **Complexity:** NTH
-

Total Operations

Combining all parts, the total complexity is:

$$NTDH + TH + NTH$$

The dominant term is the convolution itself:

$$NTH$$

Note: This direct convolution is **significantly less efficient** than the recurrence's NTH complexity. The *purpose* of the convolution view is to unlock the use of the **Fast Fourier Transform (FFT)**. An FFT-based convolution would change the complexity to $NHT T + HT T$, which is much faster than both the recurrence and the direct convolution.

Annotation:

Similar to the previous question, the response is well formatted and broken into very easily understandable segments. The answer is again consistent with my own reasoning for this question.

User

Introducing structure: Diagonal Weight Matrices

We can optimize the convolution implementation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation. # Introducing structure: Diagonal Weight Matrices

We can optimize the convolution implementation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation.

Python

```
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a
    diagonal matrix.

    Args:
        W: (H, H) diagonal matrix
        T: (int) number of time steps

    Return:
        kernel of shape (H, 1, T), which can be used in depthwise convolution
    """

    #####
    ##
    #                                     TODO: Implement the kernel here
    #

    #####
    ##

    #####
    ##
```

```

#                                     END OF YOUR CODE
#

#####
##
    return kernel

```

Assistant

Here is the PyTorch implementation for `make_diag_depthwise_kernel`.

This implementation leverages the fact that W is diagonal. The kernel for a full (H, H, T) convolution would be $W^t = [w_{t1}, \dots, w_{tH}]$.

A depthwise convolution (with `groups=H`) applies a *separate* 1D filter to each channel. The t -th filter is just the t -th diagonal element of the full kernel, which is the vector of powers $w_{t1}^0, w_{t1}^1, \dots, w_{t1}^{T-1}$.

We can compute all H of these T -length kernels at once using broadcasting.

The Logic Explained

- Extract Diagonals:** First, we get the H diagonal elements of W (the 's') into a vector of shape $(H,)$.
- Create Exponents:** We create a time vector of exponents $[0, 1, 2, \dots, T-1]$ of shape $(T,)$.
- Broadcast Power:** We use `torch.pow()` (or the `**` operator) with broadcasting to compute all powers at once.
 - We reshape the diagonals to $(H, 1)$.
 - We reshape the exponents to $(1, T)$.
 - `torch.pow(diagonals, exponents)` broadcasts to shape (H, T) . The element at $[i, t]$ will be w_{i1}^t .
- Final Reshape:** The function `F.conv1d` with `groups=H` expects a kernel of shape $(H, 1, T)$. We simply `unsqueeze` dimension 1 to get this final shape.

Here is the code to fill in the function:

Python

```

import torch

def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a
    diagonal matrix.

    Args:
        W: (H, H) diagonal matrix
        T: (int) number of time steps

    Return:
        kernel of shape (H, 1, T), which can be used in depthwise convolution
    """

    #####
    ##
    #                                TODO: Implement the kernel here
    #

    #####
    ##

    # Get dimensions and device
    H = W.shape[0]
    device = W.device
    dtype = W.dtype

    # 1. Get the diagonal elements (the lambdas)
    # W.diag() extracts the diagonal as a 1D tensor of shape (H,)
    lambda_vec = W.diag()

    # 2. Get the time vector (the exponents)
    # t_vec is a 1D tensor of shape (T,)
    t_vec = torch.arange(T, device=device, dtype=dtype)

    # 3. Compute all powers at once using broadcasting
    # (H,) -> (H, 1)
    # (T,) -> (1, T)
    # (H, 1) ** (1, T) -> (H, T)
    # The resulting kernel_2d[i, t] = lambda_vec[i] ** t_vec[t]
    kernel_2d = lambda_vec.unsqueeze(1) ** t_vec.unsqueeze(0)

    # 4. Reshape to (H, 1, T) for depthwise convolution
    # (H, T) -> (H, 1, T)
    kernel = kernel_2d.unsqueeze(1)

```

```
#####
##
#                               END OF YOUR CODE
#

#####
##
    return kernel
```

User

Python

```
def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

    Args:
        W: (H, H) [diagonal in practice]
        U: (H, D)
        b: (H,)
        x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
        h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1) # (N,H,T)

    # Build kernel (H,1,T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

    #####
    ##
```



```

#                                TODO: Implement the convolution here
#
#                                Hint: Use `groups` argument
#

#####
##

#####
##
#                                END OF YOUR CODE
#

#####
##
    return h_all

```

Assistant

Here is the completed PyTorch implementation for `diag_conv_ssm_forward`.

This implementation is very similar to the full convolution, but it uses the `groups=H` argument in `F.conv1d` to tell PyTorch to perform a depthwise convolution, which is far more efficient.

The Logic Explained

1. **Input `s`** : This is the pre-computed input term $t = Ux_t + b$, permuted to shape `(N, H, T)`. In `conv1d` terms, this is `(Batch, Channels, Length)`.
2. **Kernel `kernel`** : This is the T -length filter for each of the H channels, shaped `(H, 1, T)`. `kernel[i, 0, t]` contains the t -th power of the i -th diagonal element of W .
3. **Flip for Convolution**: As before, `F.conv1d` computes a cross-correlation. To get the convolution $h_{t+1} = \sum_{t=0}^t$, we must flip the kernel along its time dimension (dim 2).
4. **Pad for Causality**: To ensure the output at time t only depends on inputs from time 0 to t , we pad the input with $T - 1$ zeros on the **left**.
5. **`groups=H`** : This is the key.
 - `s` has H input channels.
 - `kernel` has H output channels.
 - `groups=H` tells `F.conv1d` to split the input and kernel into H groups. Group consists of:
 - Input channel (shape `(N, 1, T)`)

- Kernel (shape (1, 1, T))
- It then computes a 1D convolution for each group independently and stacks the results.
This exactly matches the desired operation: $h_{t+1} = , , , .$

6. **Permute Output:** The `conv1d` operation returns (N, H, T). We permute it to (N, T, H) to match the docstring.

Here is the code to fill in the function:

Python

```
import torch
import torch.nn.functional as F

# (Assuming make_diag_depthwise_kernel is defined above)

def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

    Args:
        W: (H, H) [diagonal in practice]
        U: (H, D)
        b: (H,)
        x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
        h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]

    # s = U x + b, shape (N, T, H)
    s = x @ U.T + b

    # s = (N, H, T)
    s = s.permute(0, 2, 1)

    # Build kernel (H, 1, T) for depthwise conv
    # Note: Since W is diagonal,  $W = W.T$ , so we don't have the
    # W vs W.T ambiguity from the general case.
```

```

    kernel = make_diag_depthwise_kernel(W, T)

#####
##
#           TODO: Implement the convolution here
#
#           Hint: Use `groups` argument
#

#####
##

# 1. Flip the kernel for convolution (same as before)
# (H, 1, T)
kernel_flipped = torch.flip(kernel, dims=[2])

# 2. Pad the input s for causal convolution (same as before)
# (N, H, T) -> (N, H, 2T - 1)
s_padded = F.pad(s, (T - 1, 0))

# 3. Apply 1D depthwise convolution
# input: (N, C_in, L_in) = (N, H, 2T - 1)
# weight: (C_out, C_in/groups, K) = (H, 1, T)
# groups: H
# C_in = H, C_out = H. This is a depthwise conv.
# output: (N, C_out, L_out) = (N, H, T)
h_conv = F.conv1d(s_padded, kernel_flipped, groups=H)

# 4. Permute output
# (N, H, T) -> (N, T, H)
h_all = h_conv.permute(0, 2, 1)

#####
##
#           END OF YOUR CODE
#

#####
##
    return h_all

```

User

Optimizing the recurrent implementation

Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the W matrix. # Optimizing the recurrent implementation

Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the W matrix.

Python

```
def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
        h_{t+1} = W h_t + U x_t + b
    but W is diagonal, i.e. W = diag(diagW).

    Args:
        W: (H, H) diagonal matrix
        U: (H, D)
        b: (H,)
        x: (N, T, D) => batch=N, time steps=T, input dim=D

    Returns:
        h_all: (N, T, H)
    """

    #####
    ##
    #         TODO: Implement the optimized recurrent pass here
    #

    #####
    ##

    #####
    ##
    #                               END OF YOUR CODE
    #

    #####
    ##
    return h_all
```

Assistant

Here is the optimized PyTorch implementation for the recurrent forward pass, leveraging the diagonal structure of the W matrix.

The Logic Explained

1. Original Recurrence: The general (unrolled) update rule from the first problem was:

$$h_t = F.\text{linear}(h_t, W) + F.\text{linear}(x_t, U) + b$$

The $F.\text{linear}(h_t, W)$ term computes $h_t @ W.T$ and is the most expensive part, with a cost of $N \cdot H$ operations at each time step.

2. **The Optimization:**

- We are given that W is a diagonal matrix.
- For a diagonal matrix, $W = W.T$.
- The matrix multiplication $h_t @ W$ (where h_t is (N, H) and W is (H, H) diagonal) simplifies.
- The $-th$ column of the output is just the $-th$ column of h_t multiplied by the $-th$ diagonal element of W (i.e., W).
- This is an **element-wise multiplication**, not a full matrix multiplication.
- In PyTorch, we can get the diagonal elements as a vector of shape $(H,)$ using `W.diag()`.
- The operation $h_t @ W.T$ can be replaced by $h_t * W.diag()$. PyTorch's broadcasting handles this automatically.
- This optimization reduces the cost of this term from $N \cdot H$ to just $N \cdot H$.

3. New Update Rule: The update inside the loop becomes:

$$h_t = h_t * W_diag + F.\text{linear}(x_t, U) + b$$

Here is the code to fill in the function:

Python

```
import torch
import torch.nn.functional as F

def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
        h_{t+1} = W h_t + U x_t + b
    """
```

but W is diagonal, i.e. $W = \text{diag}(\text{diag}W)$.

Args:

W: (H, H) diagonal matrix

U: (H, D)

b: (H,)

x: (N, T, D) => batch=N, time steps=T, input dim=D

Returns:

h_all: (N, T, H)

"""

#####

##

TODO: Implement the optimized recurrent pass here

#

#####

##

Get dimensions

N, T, D = x.shape

H = W.shape[0]

Get device and dtype from input

device = x.device

dtype = x.dtype

--- The Optimization ---

Extract the diagonal of W as a 1D tensor (shape H,).

We do this once, outside the loop.

W_diag = W.diag()

Pre-allocate the output tensor

h_all = torch.zeros((N, T, H), device=device, dtype=dtype)

Initialize h_0

h_t = torch.zeros((N, H), device=device, dtype=dtype)

Loop over time

for t in range(T):

 # Get the input for the current time step

 x_t = x[:, t, :] # Shape (N, D)

 # Apply the *optimized* update rule

```

# The expensive F.linear(h_t, W) or (h_t @ W.T)
# is replaced with an efficient element-wise multiply.
recurrent_term = h_t * W_diag
input_term = F.linear(x_t, U)

h_t = recurrent_term + input_term + b

# Store the new hidden state
h_all[:, t, :] = h_t

```

```

#####
##
#                                     END OF YOUR CODE
#
#####
##
    return h_all

```

Annotation:

For the GPU notebook, Gemini Pro one shots both coding portions and produces clean, commented code and a very detailed breakdown associated with the code line by line. I pasted all questions part by part to check the output for each part but I did note that it might prioritize more recent information over earlier information/context surrounding the assessment.

Conclusions:

Overall, Gemini Pro got nearly all the questions one shot all the coding portions of this assignment except one and it quickly recognized its mistake with some prompting from me. However, it took a significant amount of time to answer any questions and even longer when generating any code. The tradeoff here is that answers are very detailed when compared to other similar models and fairly accurate. The model offers explanations that are well formatted and match directly to the code and code is commented to establish correspondence with its written explanations.